

```

1  import os
2  import re
3  import csv
4  import numpy as np
5  import torch
6  import torch.nn as nn
7
8  from torch.utils.data import Dataset, DataLoader, random_split
9  from sklearn.preprocessing import StandardScaler
10 import matplotlib.pyplot as plt
11 from sklearn.metrics import confusion_matrix, classification_report, precision_score,
    recall_score, f1_score
12 from torch.utils.data import random_split
13 from config_reader import ConfigReader
14
15 #####
16 # Step 2: Dataset with sliding windows
17 #####
18 class WifiCSIDataset(Dataset):
19     def __init__(self, logger, file_list, window_size=128, stride=64, scaler_meta=None
    , scaler_csi=None):
20         self.samples = []
21         self.window_size = window_size
22         self.stride = stride
23         self.logger = logger
24         # Fit scalers if not provided
25         if scaler_meta is None:
26             self.scaler_meta = StandardScaler()
27         else:
28             self.scaler_meta = scaler_meta
29         if scaler_csi is None:
30             self.scaler_csi = StandardScaler()
31         else:
32             self.scaler_csi = scaler_csi
33
34         # First pass: collect all data for scaling
35         all_meta, all_csi = [], []
36         i = 0
37         for f in file_list:
38             X_meta, X_csi, _, _, _ = self.load_csv_as_numpy(f)
39             self.logger.info(f"A_00_{i}. X_meta: {X_meta.shape} X_csi: {X_csi.shape}")
40             all_meta.append(X_meta)
41             all_csi.append(X_csi)
42             i = i+1
43
44         self.logger.info(f"A_01. all_meta: {(len(all_meta[0]))} all_csi: {len(all_csi)}")
45
46         all_meta = np.vstack(all_meta)
47         all_csi = np.vstack(all_csi)
48         self.scaler_meta.fit(all_meta)
49         self.scaler_csi.fit(all_csi)
50         self.logger.info(f"A_02. all_meta: {all_meta.shape} all_csi: {all_csi.shape}")
51
52         # Second pass: windowed sequences
53         for f in file_list:
54             X_meta, X_csi, y, _, _ = self.load_csv_as_numpy(f)
55             X_meta = self.scaler_meta.transform(X_meta)
56             X_csi = self.scaler_csi.transform(X_csi)
57
58             T = len(X_meta)
59             for start in range(0, T - window_size + 1, stride):
60                 m_seq = X_meta[start:start+window_size] # (W, 12)
61                 csi_seq = X_csi[start:start+window_size] # (W, 99)
62                 self.samples.append((m_seq, csi_seq, y))
63
64     def __len__(self):
65         return len(self.samples)
66
67     def __getitem__(self, idx):
68         m_seq, csi_seq, y = self.samples[idx]
69         # Use the first subject label in the window (or last, depending on your task)

```

```

70     subject_label = y["subject"][0] # or y["subject"][-1]
71     return {
72         "metadata_seq": torch.tensor(m_seq, dtype=torch.float32), # (W, 12)
73         "csi_seq": torch.tensor(csi_seq, dtype=torch.float32), # (W, 99)
74         "label": torch.tensor(subject_label, dtype=torch.long) # shape: ()
75     }
76
77     def parse_complex(self, s):
78         if s is None or s.strip() == "":
79             return 0.0 + 0.0j # treat missing values as 0
80         try:
81             s = s.replace('+-', '-').replace('-+', '-').replace('i', 'j')
82             return complex(s)
83         except Exception:
84             # If still not parsable, default to 0
85             return 0.0 + 0.0j
86
87     def extract_S_A_numbers(self, filename):
88         """
89         Extract subject (Sxx) and activity (Axx) numbers from filename.
90         Example: 'E1_S01_C03_A03_T01.csv' -> (1, 3)
91         """
92         match = re.search(r'S(\d+).*A(\d+)', filename)
93         if match:
94             return int(match.group(1)), int(match.group(2))
95         return None, None
96
97     def load_csv_as_numpy(self, filename):
98         self.logger.info(f"B_00. Loading: {filename}")
99         with open(filename, 'r', newline='') as f:
100             reader = csv.DictReader(f)
101             cols = reader.fieldnames
102
103             csi_cols = [c for c in cols if c.startswith('csi_')]
104             meta_cols = [
105                 'timestamp_low', 'bfee_count', 'Nrx', 'Ntx',
106                 'rssi_a', 'rssi_b', 'rssi_c', 'agc',
107                 'perm_1', 'perm_2', 'perm_3'
108             ]
109
110             X_meta, X_csi = [], []
111             subj, act = [], []
112             s, a = self.extract_S_A_numbers(os.path.basename(filename))
113             for row in reader:
114                 # metadata
115                 meta_row = [float(row[c]) for c in meta_cols]
116                 # CSI as magnitudes
117                 csi_row = [abs(self.parse_complex(row[c])) for c in csi_cols]
118                 X_meta.append(meta_row)
119                 X_csi.append(csi_row)
120                 subj.append(s)
121                 act.append(a)
122
123             X_meta = np.array(X_meta, dtype=np.float32) # (T, 12)
124             X_csi = np.array(X_csi, dtype=np.float32) # (T, 99)
125             self.logger.info(f"B_01. X_meta: {X_meta.shape} X_csi: {X_csi.shape}")
126
127             self.logger.info(f"B_02. Subject: {len(subj)}, Activity: {len(act)}")
128             y = {"subject": subj, "activity": act}
129             return X_meta, X_csi, y, meta_cols, csi_cols
130
131

```